Ders 25 Genişletilmiş Notu: Dal ve Sınır ile Sırt Çantası Problemi, Gezgin Satıcı, Durma Problemi, P, NP, NP-Tam ve NP-Zor

Ders 25 Genişletilmiş Notu: Dal ve Sınır ile Sırt Çantası Problemi, Gezgin Satıcı, Durma Problemi, P, NP, NP-Tam ve NP-Zor

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dönemin Son Konusu
- 2. Dal ve Sınır ile 0-1 Sırt Çantası Problemi
- 3. Kaba Kuvvetle Karşılaştırma ve Algoritma Yaklaşımları Üzerine
- 4. Gezgin Satıcı Problemi ve Hamilton Devresi
- 5. Polinom Zamanda Çözülebilen ve Çözülemeyen Problemler
- 6. Karar Problemleri
- 7. Karar Verilebilir ve Karar Verilemez Problemler: Durma Problemi
- 8. Polinom Zamanda Çözülemeyen Ama Karar Verilebilir Problemler
- 9. Deterministik Olmayan Algoritmalar ve NP Sınıfı
- 10. Polinom İndirgeme ve NP-Tam Problemler
- 11. Soru-Cevap: P ve NP Örnekleri
- 12. NP-Zor Problemler ve Sınıfların Şekli
- Örnek Sorular
- Dersin Toparlaması

1. Dönemin Son Konusu

Bu ders dönemin son konu anlatımıdır. Hoca, bir aksilik olmazsa bu dersle birlikte dönemin bütün konularının bitmiş olacağını söyledi. Dersin ilk bölümünde bir önceki haftadan kalan dal ve sınır (branch and bound) örneği çözüldü. Bu örnek, daha önce dinamik programlamayla çözülen 0-1 sırt çantası (knapsack) problemiydi. Ardından gezgin satıcı problemine kısaca değinildi. Dersin ikinci bölümü hesaplama karmaşıklığının teorik kısmına ayrıldı: polinom zaman, karar problemleri, durma problemi (halting problem), deterministik olmayan algoritmalar, P, NP, NP-tam (NP-complete) ve NP-zor (NP-hard).

Konu anlatımı kaydın yaklaşık 00:22'sinde başlar. Ondan önceki kısım ve 01:37'den sonraki kısım ders düzeniyle ilgili konuşmalardır.

2. Dal ve Sınır ile 0-1 Sırt Çantası Problemi

2.1 Problemin Hatırlanması

Hoca önce bir önceki haftayı hatırlattı: Geri izleme (backtracking) ve dal ve sınır konuşulmuş, dal ve sınırla işlerin kişilere dağıtıldığı bir atama problemi çözülmüştü. O örnekte bir alt sınır (lower bound) kullanılmıştı. Bu örnekte ise bir üst sınır (upper bound) kullanılacak.

Sırt çantası problemi, bir eve giren hırsızın çuvalının kapasitesinin el verdiği kadar en yüksek değerli eşyaları seçmesi problemidir. Daha önce dinamik programlamayla çözülmüştü. Bu kez aynı problem dal ve sınır bakış açısıyla çözülüyor.

Örnekte 4 eşya (item) ve kapasitesi $W = 10$ kilo olan bir çanta var:

Eşya	Ağırlık w_i	Değer v_i	v_i/w_i
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

2.2 İlk Adım: Birim Ağırlık Başına Değere Göre Sıralama

Durum uzayı ağacını (state space tree) kurmaya başlamadan önce yapılması gereken bir iş var: Her eşya için ağırlık başına değer v_i/w_i hesaplanır ve eşyalar bu değere göre büyükten küçüğe sıralanır:

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq ... \geq \frac{v_n}{w_n}$$

Bu örnekte eşyalar tesadüfen zaten sıralı verilmiş: birinci eşyada kilo başına 10, ikincide 6, üçüncüde 5, dördüncüde 4 değer var.

> **Hocanın vurgusu:** Problemi çözmeye başlamadan önce ağırlık başına değeri hesaplayıp eşyaları bu sıraya koymak gerekir. Ağaca ancak bundan sonra geçilir.

2.3 Üst Sınırın Mantığı

Ağacın her düğümünde üç değer tutulur: o ana kadar alınan eşyaların toplam ağırlığı w, toplam değeri v ve üst sınır ub. Üst sınır şu soruyla hesaplanır: Çantada kalan boş yeri, sıradaki en değerli (kilo başına) eşyayla doldurabilseydim en fazla ne kadar değere ulaşırdım?

$$ub = v + (W - w) \cdot \frac{v_{i+1}}{w_{i+1}}$$

Burada $i+1$, karar verilecek sıradaki eşyadır. Bu değer gerçekte ulaşılabilir olmayabilir, çünkü eşyalar bölünemez. Ama hiçbir çözüm bu değeri geçemez, bu yüzden bir üst sınırdır.

Hoca kökteki sınırı şöyle açıkladı: Başlangıçta $w = 0$, $v = 0$. Mümkün olsaydı birinci eşyayı almak isterdik. Birinci eşya kilo başına 10 değerle en üst sırada. Çanta 10 kilo. Eğer birinci eşyadan 10 kilo alabilseydik değer 100 olurdu. Elbette bu mümkün değil, çünkü birinci eşya 4 kilo. Biz mümkün olanın peşinden gideceğiz. Her eşya için iki yol var: Ya o eşyayı alırız ya da almayız.

2.4 Durum Uzayı Ağacının Adım Adım Kurulması

Hoca düğümleri tahtada numaralandırdı. Ağaç şöyle kuruldu:

- **Düğüm 0 (kök):** $w = 0$, $v = 0$, $ub = 0 + 10 \cdot 10 = 100$.
- **Düğüm 2 (eşya 1 alınmadı):** $w = 0$, $v = 0$. Sıradaki eşya 2, kilo başına değeri 6. Bütün 10 kiloyu bununla doldurabilseydik: $ub = 0 + 10 \cdot 6 = 60$.
- **Düğüm 1 (eşya 1 alındı):** $w = 4$, $v = 40$. Değeri $4 \cdot 10$ diye de, eşyanın toplam değeri diye de hesaplayabilirsiniz, aynı şey. Geriye $10 - 4 = 6$ kilo kaldı. Bu 6 kilo sıradaki en yüksek kilo başına değer olan 6 ile çarpılır: $6 \cdot 6 = 36$. $ub = 40 + 36 = 76$.

Hoca, 76'nın bu değerlerle oluşturulabilecek en büyük değer olduğunu, ama kısıtlara (constraint) uymadığını, sadece bir sınır olduğunu belirtti. 76 ile 60 karşılaştırılır ve sınırı daha iyi olan düğüm 1'den devam edilir.

- **Düğüm 3 (eşya 1 ve 2 alındı):** Ağırlık $4+7=11>10$. Kısıta uymuyor, uygun değil (not feasible). Bu düğümün altındaki ihtimaller elenmiş olur, peşinden gitmeye gerek kalmaz.
- **Düğüm 4 (eşya 1 alındı, 2 alınmadı):** $w=4$, $v=40$. Eşya 2 alınmadığı için kalan 6 kilo bu kez üçüncü eşyanın kilo başına değeri 5 ile çarpılır: $6\cdot 5=30$. $ub=40+30=70$.

Şimdi açık düğümler düğüm 2 ($ub=60$) ve düğüm 4 ($ub=70$). Hoca, 60'ın peşinden gitmediğini, hâlâ 70'in peşinden gittiğini söyledi. O an için hep en iyi çözümün (best solution) peşinden gidiliyor. Geride kalan düğümler, sonradan bulunan çözümlerle karşılaştırılacak ve peşlerinden gitmeye gerek olup olmadığına orada karar verilecek.

- **Düğüm 6 (eşya 3 alınmadı):** $w=4$, $v=40$. Eşya 2 ve 3 alınmadığı için kalan 6 kilo dördüncü eşyanın değeri 4 ile doldurulur: $6\cdot 4=24$. $ub=40+24=64$.
- **Düğüm 5 (eşya 3 alındı):** $w=4+5=9$, $v=40+25=65$. 1 kiloluk boşluk kaldı. Bu boşluk kalan eşya 4'ün kilo başına değeri 4 ile doldurulur: $1\cdot 4=4$. $ub=65+4=69$.

Açık düğümler: düğüm 2 (60), düğüm 6 (64), düğüm 5 (69). En iyi sınır düğüm 5'te, oradan devam edilir.

- **Düğüm 7 (eşya 4 alındı):** Ağırlık $9+3=12>10$. Uygun değil.
- **Düğüm 8 (eşya 4 alınmadı):** $w=9$, $v=65$. Başka eşya kalmadığı için sınır da 65'tir. Bu bir yapraktır.

Düğüm	Kararlar	w	v	ub
0	(kök)	0	0	100
1	1 alındı	4	40	76
2	1 alınmadı	0	0	60
3	1, 2 alındı	11	-	uygun değil
4	1 alındı, 2 alınmadı	4	40	70
5	+ 3 alındı	9	65	69
6	+ 3 alınmadı	4	40	64
7	+ 4 alındı	12	-	uygun değil
8	+ 4 alınmadı	9	65	65

2.5 Sonuç ve Budama

Düğüm 8'de değeri 65 olan uygun bir çözüm bulundu. Düğüm 2'nin üst sınırı 60'tı. Yani düğüm 2'nin altından en fazla 60 değer çıkabilir. Elimizde değeri 65 olan bir seçenek olduğu için düğüm 2'nin peşinden gitmeye gerek yok, o düğüm değerlendirme dışı kalır. Aynı mantıkla sınırı 64 olan düğüm 6 da 65'i geçemez. Optimum çözüm: **eşya 1 ve 3 alınır, toplam ağırlık 9, toplam değer 65.**

Hoca burada uygulanan yaklaşımın en iyi öncelikli dal ve sınır (best-first branch and bound) olarak anlatıldığını söyledi: Her adımda o an sınırı en iyi olan düğümden devam edilir.

Python ile doğrulama: En iyi öncelikli dal ve sınır aynı 9 düğümü aynı w, v, ub değerleriyle üretti. Bütün alt kümelerin kaba kuvvetle denenmesi de en büyük değeri 65, eşya kümesini $\{1,3\}$ olarak verdi.

2.6 Düğüm 8'den Önce Karar Verilebilir miydi?

Hoca derse bir soru sordu: Düğüm 2'nin artık bir seçenek olmadığına düğüm 8'de karar verdik. Buna 7 ve 8'e bakmadan, daha önce karar verebilir miydik? En iyi sonuca değil, ama düğüm 2'nin elendiğine?

Bir öğrenci evet dedi: Düğüm 5'te değer zaten 65. Hoca bunu onayladı. Düğüm 5'te $v=65$ olan, yani elimizde olan bir seçenek var (eşya 1 ve 3, ağırlık 9, uygun). Düğüm 2'nin üst sınırı 60. Bu yüzden düğüm 5 hesaplandığı anda, 7 ve 8'e bakmadan düğüm 2'nin bir seçenek olmaktan çıktığı görülür.

3. Kaba Kuvvetle Karşılaştırma ve Algoritma Yaklaşımları Üzerine

Hoca, 4 eşyalı problemi kaba kuvvet (brute force) yaklaşımıyla çözmeye kalksaydık kaç seçeneğe bakacağımızı sordu: $2^4 - 1 = 15$ kontrol. Dal ve sınırda ise düğümler sayılınca 9 tane yapılmış oldu. Hoca bunun en kötü ihtimal olarak söylendiğini belirtti. Eşya sayısı 5'e çıkınca kaba kuvvette $2^5 - 1 = 31$, 6'ya çıkınca $2^6 - 1 = 63$ kontrol gerekir.

Bu bir kombinatorik problemdir, yani bütün kombinasyonların işin içinde olduğu bir problem. Eşya sayısı arttıkça kaba kuvvet yaklaşımının yetersizliği ortaya çıkar. Hem dinamik programlamada hem de dal ve sınırda hedef, mümkün olduğunca bütün olasılıklara bakmadan hedefe gitmektir.

> **Ek bilgi (derste anlatılmadı):** $2^4 - 1 = 15$ sayısı boş küme dışarıda bırakılınca çıkar. Boş küme de sayılırsa $2^4 = 16$ alt küme vardır. n eşyalı bir problemde kaba kuvvetin bakacağı alt küme sayısı 2^n mertebesindedir.

Hoca, bir önceki haftanın atama problemini de çözen farklı yaklaşımlar olduğunu söyledi. Bunlardan biri Macar algoritmasıdır (Hungarian algorithm). Hoca bunun bu problemleri polinom zamanda çözebilen bir algoritma olduğunu, vakit olursa genel kültür için bakılmasını tavsiye etti.

Hoca ayrıca derste anlatılan algoritmik yaklaşımları farklı problemler üzerinde çalışmayı önerdi: böl ve yönet (divide and conquer), azalt ve yönet (decrease and conquer), dönüştür ve yönet (transform and conquer), dinamik programlama, dal ve sınır, çizge yapısı kullanılan çözümler. Genişlik öncelikli arama (breadth-first search) ve derinlik öncelikli arama (depth-first search) gibi yöntemlerin hangi tür problemlerde kullanıldığını, bir problemin kaç farklı yaklaşımla çözülebildiğini ve hangisinin daha verimli olduğunu öğrenmek faydalıdır.

> **Hocanın vurgusu:** Google, Apple, Microsoft gibi uluslararası şirketlerin işe alımlarında veri yapıları, algoritmalar ve algoritma analizi konularında sıkı mülakatlar yapılır. Sorular çok zor değildir, ama bu konulara önceden göz atmayı gerektiren bir hazırlık ister.

4. Gezgin Satıcı Problemi ve Hamilton Devresi

Gezgin satıcı problemi (Traveling Salesman Problem, TSP), bir satıcının n tane şehre uğraması gerektiği ve bu şehirlere en az zaman harcayarak, en kısa yoldan ulaşmak istediği durumdan ortaya çıkmış bir problemdir. Hoca, TSP'nin NP-tam problemlerden biri olduğunu ve bunun ne demek olduğunun dersin ikinci bölümünde konuşulacağını söyledi.

TSP, Hamilton devresi (Hamiltonian circuit) problemiyle yakından ilgilidir. Hamilton devresi problemi şunu sorar: Verilen bir çizgede (graph) A noktasından başlayıp diğer bütün noktalara uğrayarak yine A noktasına dönen bir yol var mı? Böyle birden fazla devre olabilir. Gezgin satıcı bunların en kısa olanını arar. Hamilton devresi ise böyle bir yolun olup olmadığının peşindedir. Hoca aralarındaki en önemli farkı böyle özetledi.

TSP de dal ve sınır yaklaşımıyla çözülebilir. Bunun için her düğümün kendisine en yakın iki komşusuna olan uzaklıkları alınır. Bu uzaklıkların ortalaması, yani bütün düğümler için toplamın yarısı, olası en kısa turun bir sınırı olarak kullanılır. Algoritma bu sınırın hangi noktasına kadar gidilebildiğini bulmaya çalışır.

> **Ders sonrası not:** Hoca bu değere önce "alt sınır" dedi, sonra "pardon, üst sınır" diye düzeltti. Doğrusu alt sınırdır (lower bound). TSP bir minimizasyon problemidir ve her turda her şehir iki kenarla bağlandığı için, en yakın iki komşu uzaklıklarının toplamının yarısı hiçbir turun uzunluğunu geçemez. Bu durum rastgele küçük örneklerde python ile denendi: Sınır her örnekte optimum tur uzunluğundan küçük ya da ona eşit çıktı.

> **Ders sonrası not:** Hoca TSP'nin "Hamilton devresi probleminin bir alt problemi" olduğunu söyledi. Daha doğrusu TSP, Hamilton devresinin bir genellemesidir: "Hamilton devresi var mı?" sorusuna "en kısası hangisi?" sorusu eklenir. Hoca ayrıca dal ve sınırın "maksimizasyon problemleri için" ortaya atıldığını söyledi. Dal ve sınır genel olarak optimizasyon problemleri içindir: Bir önceki dersteki atama problemi bir minimizasyon, bu dersteki sırt çantası bir maksimizasyon problemidir.

5. Polinom Zamanda Çözülebilen ve Çözülemeyen Problemler

Hoca bu kısmın teorik olduğunu, bir bölümünün rahat anlaşıldığını ama bir noktadan sonra karışıklaştığını söyledi. Hayatımızda polinom zamanda çözülebilen ve çözülemeyen problemler vardır. Hoca bunları üç tipe ayırdı:

- Polinom zamanda çözülebilen problemler. Bu derste görülen algoritmaların büyük çoğunluğu böyledir.
- Polinom zamanda çözülemeyen ama çözümü olan problemler. Yeterince güçlü bir cihaz ya da yeterince zaman olsaydı çözülebilirlerdi.
- Hoca üçüncü grubu, yeterince güçlü bir cihaz ve zaman olsa bile çözülemeyecek problemler olarak tanımladı ve bunlara NP-zor dedi. Bu tanım yanlıştır; düzeltmesi Bölüm 12'dedir.

İkinci gruba güzel bir örnek gezgin satıcı problemidir. Girdi sayısı küçükken (3, 5, 10) sorun yoktur. Ama girdi sayısı 1000 ya da 10.000 olunca durum değişir. Dijital haritaları düşünün: GPS servislerinin çalıştığı navigasyon cihazlarında on binlerce nokta vardır. Her birini hesaba katmak çok zordur. 10.000'e gerek kalmadan 100 hedef koysanız bile, kâğıt üstünde baktığınızda bu işi çözmek çok yüksek bir zaman alabilir.

Hoca, bu konuda temel bilgileri öğrenmenin yeterli olduğunu söyledi. Ayrıca karmaşıklık değerlendirmesinde genelde en kötü durumun (worst case) referans alındığını hatırlattı.

5.1 Tanım 1: Tractable ve Intractable

Tanım 1: Polinom zamanda (polynomial time) çözülebilen problemler tractable, polinom zamanda çözülemeyen problemler intractable olarak adlandırılır.

Polinom zaman derken, Big-O cinsinden ifade ettiğimiz $O(n)$, $O(\log n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$ gibi karmaşıklıklar kastedilir. Hocaya göre pratikte n^4'ün üzerinde bir polinom derecesiyle genelde karşılaşılmaz. Bu önemli bir noktadır. Polinomlarda kullanılan katsayılar da genelde çok büyük olmaz. n ile kıyaslandığında, problemi normal bir sürede çözmeyi engelleyecek bir katsayı kullanımı söz konusu olmaz. Bugün bir algoritmanın hesaplama karmaşıklığı (computational complexity) bu yaklaşımlarla ifade edilir.

Polinom zamanda ifade edilemeyen karmaşıklıklar da vardır: 2^n, a^n, n^n, $n!$, $(n+1)!$, $(n+k)!$ gibi. Bunlar çok yüksek sayıda işlem gerektirir. n büyükse çok güçlü bir cihaz bile bu problemleri ancak yıllar içinde çözebilir. Bunlar tercih edilmez.

5.2 Tanım 2: P Sınıfı

Tanım 2 (P): P sınıfı, deterministik algoritmalarla polinom zamanda çözülebilen karar problemlerinin sınıfıdır.

6. Karar Problemleri

Problemlerin bir kısmı aslında karar problemidir (decision problem). Hoca, şimdiye kadar problemleri pek karar problemi olarak adlandırmadıklarını, ama bir problemi karar problemine çevirmenin bu konuları anlamayı çok kolaylaştırdığını söyledi.

Hocanın örneği çizge boyamaydı. Bir çizgenin, yan yana iki komşu aynı renkte olmayacak şekilde en az kaç renkle boyanabileceğini sormak ile çizgenin 3 renkle boyanıp boyanamayacağını sormak farklı sorulardır. Birinde sayıyı ararsınız ve bütün olasılıkları gözeterek en iyi sonuca gitmek istersiniz. Öbüründe yalnız size verilen değerin mümkün bir çözüm olup olmadığına karar verirsiniz ve cevap evet ya da hayırdır.

Bu yüzden polinom, polinom olmayan ve deterministik olmayan polinom problemler anlatılırken önce problemler karar problemi olarak formüle edilir.

> **Hocanın vurgusu:** Problemleri karar problemi olarak ele almak bu konuyu anlatmak için çok önemlidir.

7. Karar Verilebilir ve Karar Verilemez Problemler: Durma Problemi

Arada hoca kısaca tekrar etti: Polinom zamanda çözülebilen problemler tractable, çözülemeyenler intractable olarak adlandırılmıştı. Bir problemi karar problemine indirgediğinizde iki seçenekle karşı karşıyasınız: Problem ya karar verilebilir (decidable) ya da karar verilemez (undecidable) olur.

Birçok problem aslında karar verilebilirdir. Hamilton devresi probleminde, verilen bir kümeyle bir noktadan başlayıp bütün noktalara uğrayarak dönülüp dönülmediğini ortaya koyabilirsiniz. Gezgin satıcı probleminde de yeterince süre olduğunda verilen bir seçeneğin çözüm olup olmadığını ortaya koyabilirsiniz.

Bir de karar verilemez, yani ne kadar zamanınız olursa olsun sonucunu bulamayacağınız problemler vardır. Hoca, karar verilemez diye anlatılan başka problemler de olduğunu, ama en rahat anlaşılan ve anlatılan örneğin durma problemi olduğunu söyledi.

7.1 Durma Problemi

Bir bilgisayar programı ve ona verilen bir girdi düşünün. Soru şudur: Bu program bu girdiyle bir sonuç üretip durur mu, yoksa sonsuz döngüye mi girer? Buna karar vermeniz isteniyor.

7.2 Çelişkiyle İspat

Hoca ispatı tahtada adım adım yazdı. Altyazıda ispatın bazı satırları kopuk geçiyor. Aşağıdaki akış, hocanın söylediği parçalardan kurulmuştur (ara adım eklendi).

- **Varsayım:** Bu problemi çözen bir A algoritması olsun. Bir P programı ve bir I girdisi için:
  - $A(P, I) = 1$, eğer P programı I girdisiyle durursa.
  - $A(P, I) = 0$, eğer P programı I girdisiyle durmazsa.
- **Programa kendisini girdi olarak vermek:** Yeni bir Q programı tanımlanır. Q, girdi olarak bir P programı alır:
  - $A(P, P) = 0$ ise, yani P programı P girdisiyle durmuyorsa, $Q(P)$ durur (halts).
  - $A(P, P) = 1$ ise, yani P programı P girdisiyle duruyorsa, $Q(P)$ durmaz.
- P **yerine** Q **koymak:** Hocanın deyişiyle "Q'yu P ile substitute edelim", yani Q kendisine girdi olarak verilir:
  - $Q(Q)$ durur, ancak ve ancak $A(Q, Q) = 0$ ise. Bu da "Q programı Q girdisiyle durmaz" demektir.
  - $Q(Q)$ durmaz, ancak ve ancak $A(Q, Q) = 1$ ise. Bu da "Q programı Q girdisiyle durur" demektir.

Bir tarafta durduğunu varsaydığınız bir sonuç var, ama koşul "Q durmazsa" diyor. Diğer tarafta da "Q, Q girdisiyle durursa program durmayacaktır" diyorsunuz. Bu bir çelişkidir (contradiction). Demek ki böyle bir A algoritması yoktur.

Hoca, bunun Alan Turing tarafından karar verilemez problem olarak ortaya atıldığını söyledi. Yani bir bilgisayar programının verilen bir girdi için sonsuz döngüde mi devam edeceği, yoksa bir sonuç mu üreteceği sorusunun cevabı karar verilemezdir. Hoca bu tür problemlerle çok karşılaşılmayacağını, ama özellikle bazı matematiksel ifadelerde bunu bilmenin önemli olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Durma probleminin karar verilemez olduğunu Alan Turing büyük olasılıkla 1936 yılındaki makalesinde göstermiştir. İspat, Cantor'un köşegen yönteminin (diagonalization) bir uygulamasıdır: Varsayılan A algoritması, kendisini girdi olarak alan bir programla kendi cevabıyla çelişmeye zorlanır.

8. Polinom Zamanda Çözülemeyen Ama Karar Verilebilir Problemler

Polinom zamanda çözülemeyen, ama karar problemi olarak sorulduğunda cevabı alınabilen problemler vardır. Hocanın verdiği örnekler:

- **Hamilton devresi problemi:** Verilen bir çizgenin Hamilton devresi var mı, yok mu? (Determine whether a given graph has a Hamiltonian circuit.)

- **Sırt çantası problemi:** Normalde polinom zamanda çözülemeyen bir problemdir. Ama küçük ve uygun girdiler için dal ve sınırla ya da dinamik programlamayla çözülebilir. Asıl konuşulan n'nin büyük olduğu durumlardır.
- **Bölüntü problemi (partition problem):** Pozitif tam sayılardan oluşan bir küme, toplamları eşit iki ayrık kümeye ayrılabilir mi? Hoca bunun deterministik olmayan polinom problemler arasında geçtiğini ve polinom zamanda çözülemediğini söyledi.
- **Çizge boyama problemi (graph coloring)**
- **Tam sayılı doğrusal programlama (integer linear programming)**

  Ders sonrası not: Bu problemlerin "polinom zamanda çözülemediği" kanıtlanmış değildir. Kanıtlanmış olan, bugüne kadar bunlar için polinom zamanlı bir algoritma bulunamamış olmasıdır. Bu, $P = NP$ sorusunun açık olmasıyla aynı şeydir.

  Ek bilgi (derste anlatılmadı): Sırt çantası ve bölüntü problemlerinin $O(nW)$ ya da $O(n \cdot \text{toplam})$ süreli dinamik programlama çözümleri vardır. Bu süre girdideki sayıların büyüklüğüne bağlıdır, girdinin bit uzunluğuna göre polinom değildir. Bu tür algoritmalara sözde polinom (pseudo-polynomial) denir. Bu yüzden dersteki DP çözümü, bu problemlerin P'de olduğunu göstermez.

9. Deterministik Olmayan Algoritmalar ve NP Sınıfı

9.1 Tanım 3: Deterministik Olmayan Algoritma

Hoca tanımı İngilizce olarak tahtaya yazdı ve Türkçe açıkladı.

Tanım 3: Deterministik olmayan bir algoritma (nondeterministic algorithm), girdi olarak bir karar probleminin bir I örneğini (instance) alan ve iki aşamada çalışan bir yordamdır (two-stage procedure):

- **Deterministik olmayan aşama (nondeterministic stage), diğer adıyla tahmin aşaması (guessing stage):** Rastgele bir S dizgisi (string) üretilir. S, verilen örnek için aday bir çözüm (candidate solution) olarak düşünülebilir. Hoca, bunun gerçekten bir çözüm olma ihtimali de olduğunu, tamamen yanlış ve anlamsız bir çözüm olma ihtimali de olduğunu söyledi.
- **Deterministik aşama (deterministic stage), diğer adıyla doğrulama aşaması (verification stage):** Deterministik bir algoritma hem I'yı hem de S'yi girdi olarak alır. S, I örneğinin bir çözümüyse "evet" (yes) çıktısını verir. S, I'nın çözümü değilse algoritma ya "hayır" (no) döndürür ya da hiç durmayabilir (is allowed not to halt at all).

Deterministik olmayan bir algoritmanın doğrulama aşaması polinom zamanlıysa, bu algoritmaya deterministik olmayan polinom (nondeterministic polynomial) algoritma denir. Hoca bunu tekrar etti: İlk aşama, yani tahmin aşaması deterministik değildir. Ama verilen I ve S girdileri için "S, I'nın çözümü mü?" sorusu polinom zamanda doğrulanabiliyorsa, algoritma deterministik olmayan polinomdur.

9.2 Tanım 4: NP Sınıfı

Tanım 4 (NP): NP sınıfı, deterministik olmayan polinom algoritmalarla çözülebilen karar problemlerinin sınıfıdır (Class NP is the class of decision problems that can be solved by nondeterministic polynomial algorithms). Bu problem sınıfına deterministik olmayan polinom (nondeterministic polynomial) denir.

Birçok karar problemi NP sınıfına girer. NP problemlerinin içinde P problemleri de yer alır, yani NP sınıfı P sınıfını kapsar:

$$P \subseteq NP$$

Ama $P = NP$ olup olmadığı hâlâ bir soru işaretidir. Bu teorik olarak henüz ispatlanmış bir şey değildir.

9.3 Doğrulama Aşamasının Önemi

Hoca, deterministik olmayan polinom derken bir problemin doğrulama aşamasının polinom olması gerektiğini tekrar vurguladı. Örnek olarak gezgin satıcı, Hamilton devresi ve çizge boyama problemlerini verdi. Çizge boyamada "en az kaç renkle boyanabilir?" sorusunun cevabını polinom zamanda veremezsiniz. Ama problemi karar problemine dönüştürüp "çizgemi 3 renkle boyasam bu olur mu?" diye sorduğunuzda evet ya da hayır cevabı verebilirsiniz. Artık size verilen bir çözümü doğrulama şansına sahip olursunuz.

> **Hocanın vurgusu:** Burası önemli: doğrulama aşaması (verification stage).

10. Polinom İndirgeme ve NP-Tam Problemler

10.1 Tanım 5: Polinom İndirgeme

Tanım 5: Bir D_1 karar problemi, şu şartları sağlayan ve D_1'in örneklerini D_2'nin örneklerine dönüştüren bir t fonksiyonu varsa, D_2 karar problemine polinom zamanda indirgenebilir (polynomially reducible) denir ($D_1 \leq_p D_2$):

- t, D_1'in bütün evet örneklerini D_2'nin evet örneklerine, D_1'in bütün hayır örneklerini D_2'nin hayır örneklerine götürür.
- t, polinom zamanlı bir algoritmayla hesaplanabilir (computable by a polynomial time algorithm).

Hoca tanımı şöyle açıkladı: Bir D_1 probleminin D_2'ye polinom zamanda indirgenebilmesi için, D_1'in aldığı bütün evet cevaplarının D_2 içinde de geçerli olması, bütün hayır örneklerinin de D_2'de geçerli olması ve t fonksiyonunun polinom zamanda bir algoritmayla hesaplanabilmesi gerekir.

10.2 Tanım 6: NP-Tam

Tanım 6: Bir D karar problemi şu iki şartı sağlıyorsa NP-tamdır (NP-complete):

- D, NP sınıfına aittir.
- NP'deki her problem D'ye polinom zamanda indirgenebilir.

Hoca bunu bir şekille anlattı: Sizin ortaya attığınız bir problem var. NP problemleri uzayındaki bütün problemler sizin bu probleminize indirgenebiliyorsa, işte o zaman bu probleme NP-tam denir. Bir problemin NP-tam olabilmesi için yalnız NP'de yer alması yetmez; NP problemlerinin hepsinin bu probleme polinom zamanda indirgenebilmesi de gerekir.

10.3 Hamilton Devresi ve Gezgin Satıcı Örneği

Hoca NP-tamlık için en güzel örneğin Hamilton devresi ile gezgin satıcı problemi olduğunu söyledi. Bir tarafta Hamilton devresi problemini çözerken bir noktadan başlayıp çizgenin aynı noktasına dönen bir devre olup olmadığını kontrol edersiniz. Öbür tarafta TSP'de bunun en kısasını bulmaya çalışırsınız. Hoca "gezgin satıcı problemini Hamilton devresi problemine indirgeme şansına sahipsiniz" dedi ve bunlar birbirine indirgenebilen problemler oldukları için NP-tam olarak tarif edildiklerini söyledi.

Burada kritik olan, bu problemlerin doğrulama aşamasıdır. Bir çizge ve bir aday devre verildiğinde, bunun Hamilton devresi özelliği taşıyıp taşımadığına evet ya da hayır cevabı verilebilir. Ama bütün Hamilton devrelerini bulmak, çizgedeki bütün olasılıkları denemek polinom zamanda mümkün olmadığı için bu problem NP problemleri arasında gösterilir.

> **Ders sonrası not:** Hoca "TSP'yi Hamilton devresi problemine indirgeyebiliyoruz, o yüzden NP-tamdır" dedi. İndirgemenin yönü ters. NP-tamlığı göstermek için bilinen bir NP-tam problem, yeni probleme indirgenir. Hamilton devresi problemi TSP'nin karar biçimine şöyle indirgenir: n düğümlü çizgede kenar olan çiftlere 1, kenar olmayan çiftlere 2 uzaklık verilir. Uzunluğu en fazla n olan bir tur ancak ve ancak çizgede Hamilton devresi varsa vardır. Bu dönüşüm polinom zamanlıdır. TSP'nin karar biçimi NP'de olduğu ve NP-tam bir problem ona indirgendiği için NP-tamdır. Bu eşdeğerlik 6 düğümlü rastgele çizgelerde python ile denendi ve her örnekte doğru çıktı.

11. Soru-Cevap: P ve NP Örnekleri

Soru: Bu konu sözel bir konu mu, yoksa örnekler olacak mı?

Cevap: Hoca örneklerle açıkladı. Bir dizinin en büyük ya da en küçük elemanını bulmak polinom zamanda yapılabilir. Hoca, bir matrisi başka bir matrisle çarpmanın karmaşıklığını öğrenciye sordu; öğrenci n^3 dedi. n ne kadar büyük olursa olsun bu da polinom zamanda hesaplanabilir. Bu problemler P'dedir.

Buna karşılık bölüntü problemi şunu sorar: Pozitif sayılardan oluşan bir küme öyle bir yerden ikiye ayrılsın ki bir taraftaki sayıların toplamı diğer taraftakilerin toplamına eşit olsun; böyle bir çözüm var mı? Bu normalde bütün kombinasyonları gerektirir: 2^n mertebesinde bölünmenin hepsi denenir ve her biri için iki tarafın toplamı alınır. n 1000 olduğunda bunun polinom zamanda çözülme ihtimali yoktur. Hoca bunun bir NP, yani deterministik olmayan polinom problem olduğunu söyledi.

Ama bu problemin bir doğrulama aşaması vardır. Hoca öğrenciye bunun ne olduğunu sordu. Öğrenci "eşit bölündü mü diye sorarsak" dedi. Hoca bunu şöyle düzeltti: "Eşit bölündü mü" demeyelim. Şu seti verdiğimizde, şu noktadan böldüğümüzde bu problemin koşulu sağlanıyor mu, sağlanmıyor mu? Bölme orta noktadan olabilir. Ya da 3 elemanı bir kümeye, 8 elemanı başka bir kümeye koyabilirsiniz. Ya da 1000 elemanı bir kümeye, 2000 elemanı başka bir kümeye. Problemi böyle verdiğinizde, "bu bir çözüm müdür?" sorusuna bir cevap döner: Toplanır, eşit mi diye bakılır, ya evet ya hayır döner.

Bu evet veya hayır cevabı polinom zamanda verilebildiği için problemin doğrulama aşaması polinom zamanda çözülür ve problem bir NP problem olur. Problemin kendisini polinom zamanda çözemiyoruz, ama probleme verdiğimiz bir girdinin evet mi hayır mı olduğunu söyleyebiliyoruz.

Hoca bunu NP-tamlığa bağladı: NP-tam kısmında problemler polinom olarak belirli problemlere dönüştürülür. Örneği yine TSP ve Hamilton devresiydi. Hoca bunun çok küçük bir örnek olduğunu, konunun çok daha farklı örnekleri olduğunu ve daha fazla zaman ayırmak gerektiğini ekledi.

> **Ek bilgi (derste anlatılmadı):** NP-tam olduğu bilinen ilk problem, mantık formüllerinin sağlanabilirliği (SAT) problemidir (Cook-Levin teoremi, büyük olasılıkla 1971). Bundan sonra bir problemin NP-tam olduğunu göstermek için NP'deki bütün problemleri tek tek indirgemek gerekmez: Problemin NP'de olduğunu göstermek ve bilinen bir NP-tam problemi ona polinom zamanda indirgemek yeterlidir. Hamilton devresi, bölüntü, sırt çantası (karar biçimi) ve çizge boyama (3 ve üstü renk) NP-tam problemlerdir.

12. NP-Zor Problemler ve Sınıfların Şekli

12.1 Hocanın NP-Zor Anlatımı

Hoca, NP-tamın ötesinde NP-zor (NP-hard) problemler olduğunu söyledi. Örnek olarak az önceki durma problemini verdi ve NP-zor problemleri "polinom zamanda çözülemeyen, aynı zamanda doğrulaması da polinom zamanda yapılamayan" problemler olarak tarif etti. Hocaya göre doğrulamayı polinom zamanda yapamıyorsanız problem NP-zordur. Doğrulamayı polinom zamanda yapabiliyorsanız ve NP'deki diğer problemler bu probleme indirgenebiliyorsa problem NP-tamdır.

Hoca daha sonra şunu ekledi: Durma problemi NP-zor bir problemdir. Hem tahmin aşaması hem doğrulama aşaması, bir soru sorduğumuzda polinom zamanda çözülemez. Doğrulama aşaması çözülebiliyorsa problem NP'dedir. Doğrulama aşaması çözülebiliyor ve NP havuzundaki problemler ona indirgenebiliyorsa problem NP-tamdır.

> **Ders sonrası not:** Hocanın NP-zor tanımı yanlıştır. NP-zorun tanımı: NP'deki her problem kendisine polinom zamanda indirgenebilen problem NP-zordur. Problemin NP'de olması gerekmez, ama "doğrulaması polinom zamanda yapılamayan" ya da "ne kadar zaman olsa da çözülemeyen" diye bir şart yoktur. NP-zor bir problem karar verilebilir de olabilir: Örneğin TSP'nin optimizasyon biçimi NP-zordur ve sonlu sürede çözülebilir. Her NP-tam problem de NP-zordur. Durma probleminin NP-zor

olduğu doğrudur (NP'deki her problem ona indirgenebilir), ama nedeni karar verilemez olması değil, bu indirgemenin var olmasıdır.

12.2 Sınıfların Şekli

Hoca tanımları bir kez daha gözden geçirmeyi önerdi: polinom problemler (P), deterministik olmayan polinom problemler (NP), NP-tam problemler ve NP-zor problemler. Bu kısmın önemli olduğunu söyledi.

Hoca sonra bu sınıfların sık karşılaşılan şeklini çizdi:

- NP problemleri, NP-tam ve polinom problemleri kapsar. P, NP'nin içindedir.
- NP-tam problemler aynı zamanda NP-zor havuzunun da bir parçasıdır. Yani NP-tam problemler, NP ile NP-zorun kesişimidir.
- Bu şekil, P ile NP'nin aynı olmadığı varsayımıyla çizilir. Bunların eşit olduğu senaryo için de bir gösterim vardır, ama daha çok bu şekil kullanılır.

Buradan şu çıkar: NP-tam bir problem NP-zor olarak da sayılabilir. Ama NP-zor bir problemin NP-tam olacağının bir garantisi yoktur. Hoca bunun, o problemin doğrulama aşamasının polinom zamanda çözülemediği anlamına geldiğini söyledi.

> **Hocanın vurgusu:** Doğrulama aşaması çok önemlidir. Verilen bir girdi için polinom zamanda bir çıktı üretilebilmesi bir algoritma için çok çok önemlidir. Sorulması gereken en önemli soru budur.
>
> **Ders sonrası not:** "NP-zor ama NP-tam olmayan problemin doğrulaması polinom zamanlı değildir" ifadesi tam doğru değildir. NP-zor bir problem NP-tam değilse, bunun nedeni o problemin NP'de olmaması ya da NP'de olduğunun bilinmemesidir. Örneğin TSP'nin optimizasyon biçimi ("en kısa tur hangisi?") bir karar problemi olmadığı için NP'de sayılmaz; doğrulamanın zorluğu ise ayrı bir sorudur.

Örnek Sorular

Soru 1 (derste sözlü, 00:33:56): Sırt çantası örneğinde düğüm 2'nin artık bir seçenek olmadığına düğüm 8'de karar verdik. Düğüm 7 ve 8'e bakmadan, daha önce buna karar verebilir miydik?

> **Çözüm (derste verildi):** Evet. Düğüm 5 hesaplandığında elde $w = 9$, $v = 65$ olan uygun bir seçenek vardır (eşya 1 ve 3). Düğüm 2'nin üst sınırı 60'tır. $60 < 65$ olduğu için düğüm 2'nin altından daha iyi bir çözüm çıkamaz ve düğüm 2, düğüm 7 ve 8'e bakmadan elenir.

Soru 2 (derste sözlü, 00:34:44): 4 eşyalı bu problemi kaba kuvvet yaklaşımıyla çözseydik kaç seçeneğe bakacaktık? Dal ve sınırda kaç düğüm oluşturuldu?

> **Çözüm (derste verildi):** Kaba kuvvette $2^4 - 1 = 15$ kontrol yapılır. Dal ve sınırda kök dahil 9 düğüm oluşturuldu. Eşya sayısı 5 olunca kaba kuvvette 31, 6 olunca 63 kontrol gerekir.

Soru 3 (derste sözlü, 01:29:02): Bir matrisi başka bir matrisle çarpmanın karmaşıklığı nedir?

> **Çözüm (derste verildi):** $O(n^3)$. n ne kadar büyük olursa olsun polinom zamanda hesaplanır, yani problem P'dedir.

Soru 4 (derste sözlü, 01:30:01): Bölüntü probleminin doğrulama aşaması nedir?

> **Çözüm (derste verildi):** Kümenin belirli bir bölünmesi verilir (örneğin bazı elemanlar bir kümeye, geri kalanı diğerine). İki kümenin elemanları toplanır ve toplamların eşit olup olmadığına bakılır. Cevap evet ya da hayırdır ve bu kontrol polinom zamanda yapılır. Bu yüzden bölüntü problemi NP'dedir.

Dersin Toparlaması

Dersin ilk bölümünde 0-1 sırt çantası problemi en iyi öncelikli dal ve sınırla çözüldü. Eşyalar önce v_i/w_i değerine göre sıralandı. Her düğümde ağırlık, değer ve kalan kapasitenin sıradaki

eşyanın birim değeriyle doldurulmasından gelen üst sınır hesaplandı. Kapasiteyi aşan düğümler uygun değil diye kesildi. Sınırı bulunan en iyi çözümün altında kalan düğümler elendi. Sonuçta 15 alt küme yerine 9 düğümle optimum değer 65 (eşya 1 ve 3) bulundu. Ardından gezgin satıcı probleminin Hamilton devresiyle ilişkisi ve en yakın iki komşu uzaklığıyla kurulan sınırla dal ve sınıra uygun olduğu anlatıldı.

İkinci bölümde problemler karar problemi olarak ele alındı. Polinom zamanda çözülebilenler tractable, çözülemeyenler intractable diye adlandırıldı ve P sınıfı tanımlandı. Durma probleminin karar verilemez olduğu çelişkiyle gösterildi. Deterministik olmayan algoritmanın tahmin ve doğrulama aşamaları, NP sınıfı, polinom indirgeme ve NP-tamlık tanımlandı. Hocanın en çok vurguladığı nokta doğrulama aşamasıydı: Bir problem için verilen aday çözüm polinom zamanda doğrulanabiliyor mu? Bu dersle dönemin konu anlatımı tamamlandı. Bir sonraki hafta ders saatinde 4. ödevin çözümüne geçileceği söylendi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.